

4주차 공통 핸드아웃 — 공유 게이트웨이(:11500) 위에서 내 앱을 운영급으로

오늘 한 줄 컨셉: 이제 각자 모델을 띄우지 않습니다. 공유 게이트웨이가 생각 누출·부하·모델명을 자동으로 처리합니다. 내 앱의 모델 주소 한 줄만 `:11434` → `:11500` 으로 바꾸면 내 앱이 운영급이 됩니다.

이 문서는 **전원 공통**입니다. 본인 앱별 세부 가이드(개인별 문서)는 따로 받습니다. 이 공통 핸드아웃을 먼저 읽고, 개인별 가이드의 “정확한 줄번호”를 적용하세요.

오늘 할 일 (5단계)

순서	할 일	걸리는 시간
STEP 0	내 앱의 모델 주소를 <code>:11434</code> → <code>:11500</code> 으로 바꾸기 (한 줄)	2분
STEP 1	게이트웨이가 살아 있는지 <code>/health</code> 로 확인	1분
STEP 2	게이트웨이에 어떤 모델이 있는지 <code>/api/tags</code> 로 확인	1분
STEP 3	내 앱 기동 → 대표 질문 1개로 끝까지 확인	5분
STEP 4	before/after 비교 — <code>:11434</code> (raw) vs <code>:11500</code> (게이트웨이) 같은 질문	10분

공유 서버 주소(예시): `http://121.138.151.6:11500` 실제 주소는 수업 시작 때 강사가 칠판/채팅으로 알려줍니다. 이 문서의 `121.138.151.6` 은 그 주소로 바꿔서 읽으세요.

STEP 0 — 내 앱 주소 바꾸기 (공통 diff, :11434 → :11500)

여러분의 앱 코드 어딘가에 모델 서버 주소가 적힌 줄이 **딱 한두 줄** 있습니다. 거기서 **포트 번호**만 바꿉니다.

```
- OLLAMA_URL = "http://121.138.151.6:11434/api/chat"
+ OLLAMA_URL = "http://121.138.151.6:11500/api/chat"
```

임베딩(검색·RAG)을 쓰는 앱이면 임베딩 주소도 같이 바꿉니다. **두 줄 다** 바꿔야 검색까지 정상 동작합니다.

```
- OLLAMA_URL = "http://121.138.151.6:11434/api/chat"
- EMBED_URL  = "http://121.138.151.6:11434/api/embeddings"
+ OLLAMA_URL = "http://121.138.151.6:11500/api/chat"
+ EMBED_URL  = "http://121.138.151.6:11500/api/embeddings"
```

바꿀 곳 찾는 법

코드에서 **11434** 를 검색하세요. 나오는 곳을 전부 **11500** 으로 바꾸면 됩니다.

```
# 내 앱 폴더에서 - 11434가 어느 파일 몇 번째 줄에 있는지 찾기
grep -rn "11434" .
```

건드리지 말 것: `model` / `MODEL` 같은 **모델명 줄은 그대로 두세요**. (예: `qwen3.6:35b-a3b`, `qwen2.5:14b` 등 무엇이든) 게이트웨이가 모델명을 자동으로 정규화하므로, 모델명이 조금 달라도 알아서 맞춰 줍니다. 바꿀 것은 **포트 번호(11434→11500)** 뿐입니다.

본인 앱의 정확한 줄번호는? 개인별 가이드의 “2. 바뀌는 것 — URL 한 줄”에 파일명·줄번호가 적혀 있습니다. 이미 **:11500** 으로 되어 있는 앱도 있으니, 그 경우엔 “확인만” 하면 됩니다.

게이트웨이가 대신 해주는 것 (그래서 내 코드가 깔끔해짐)

내 앱마다 박아야 했던 운영 처리를, 이제 게이트웨이가 **한곳에서** 자동으로 해줍니다. 여러분은 신경 쓸 필요가 없습니다.

게이트웨이가 자동 처리하는 것	전(raw :11434)이라면 직접 해야 했던 일	이제(게이트웨이 :11500)
생각 끄기(<code>think:false</code>)	답변에 <code><think>...</code> <code></think></code> 혼잣말이 새어 나옴 → 코드로 잘라내야 함	게이트웨이가 자동으로 꺼 줌. 답변은 본문만 깔끔하게
답변 길이 상한 (<code>num_predict_cap=512</code>)	답이 끝없이 길어지거나 멈추지 않을 수 있음	안전한 상한을 자동 적용. 무한 응답·과부하 방지
모델명 정규화	코드에 적은 모델명이 서버 모델명과 정확히 같아야 함	이름이 조금 달라도 알아서 맞춰 줌
공유 부하 관리	각자 모델을 띄워 GPU 메모리 충돌	한 서버가 줄 세워 처리. 각자 안 띄워도 됨

한마디로: **운영 처리를 앱마다 복붙하는 대신 게이트웨이 한곳에 모았습니다.** 이것이 “하네스를 인프라로 끌어올린다”는 뜻입니다. 여러분은 앱 본연의 기능(프롬프트·검색·화면)에만 집중하면 됩니다.

 **동시성, 강사가 미리 검증했습니다.** 11명이 **동시에** 질문해도 게이트웨이는 죽지 않고 줄을 세워 처리합니다(실측: 전원 성공, 다운 0). 단, **응답 속도는 “답 길이”에 좌우됩니다.** 자료에 근거해 **간결히 답하라**는 시스템 프롬프트(= 좋은 하네스)가 있으면 답이 2~3문장으로 짧아져 **동시 11명도 약 10초 안에** 다 돌아옵니다. 반대로 “간결히” 규칙이 없으면 모델이 길게 늘어놓아 뒷사람이 한참 기다립니다. → **좋은 하네스는 품질뿐 아니라 공유 서버 속도까지 지킵니다.**

STEP 1-2 — 게이트웨이 점검 (/health , /api/tags)

내 앱을 켜기 전에, 게이트웨이가 살아 있는지부터 확인합니다. 터미널(명령 프롬프트)에 그대로 붙여넣으세요.

① 살아 있는지 — /health

```
curl http://121.138.151.6:11500/health
```

정상이면 아래처럼 돌아옵니다(보기 좋게 정리한 예시).

```
{
  "status": "ok",
  "num_predict_cap": 512,
  "chat_model": "qwen3.6:35b-a3b",
  "embed_model": "bge-m3:latest"
}
```

- `status: ok` → 게이트웨이 정상.
- `num_predict_cap: 512` → 답변 길이 상한(자동). 이 값이 보이면 길이 제어가 켜져 있는 것.
- `chat_model` / `embed_model` → 지금 연결된 대화용·검색용 모델 이름.
- **응답이 아예 없거나 오류면** → 내 앱을 켜도 안 됩니다. **강사에게 알려세요**(공유 서버 점검 필요).

② 어떤 모델이 올라와 있나 — /api/tags

```
curl http://121.138.151.6:11500/api/tags
```

- 사용 가능한 모델 목록이 나옵니다. `/health` 의 `chat_model` · `embed_model` 이 이 목록 안에 있으면 정상입니다.
- 내 코드의 모델명과 글자가 똑같지 않아도 괜찮습니다(게이트웨이가 정규화). 목록을 “참고”만 하세요.

임베딩(검색·RAG)을 쓰는 앱이라면 — 게이트웨이는 임베딩도 중계합니다. - `POST`

`/api/embeddings` : `prompt` (문장 1개) → `embedding` (숫자 1024개) 1벌. (단수) -

`POST /api/embed` : 여러 문장을 한 번에(배치). 본인 앱이 둘 중 어느 쪽을 쓰는지 는 개인별 가이드에 적혀 있습니다. 보통 RAG 앱은 `/api/embeddings` 를 씁니다.

STEP 3 — 내 앱 기동 + 대표 질문으로 끝까지 확인

게이트웨이가 `ok` 인 걸 확인했으면, 이제 내 앱을 켜고 질문 1개로 끝까지 흘러봅니다.

1. **앱 기동** — 본인 앱 폴더에서 평소대로 실행합니다(개인별 가이드에 정확한 명령이 있음).
예:

```
python -m uvicorn app:app --port 8000
```

또는

```
python3 server.py
```

2. **대표 질문 1개** — 화면에서 질문하거나, 터미널에서 직접 호출합니다. 본인 앱의 평소 질문 1개면 충분합니다.
3. **“생각 누출” 없는지 확인 (가장 중요)** — 답변 안에 `<think>` 나 모델의 혼잣말(추론 과정)이 섞여 나오면 안 됩니다.
 - 게이트웨이(`:11500`)가 `think:false` 를 자동으로 넣어 주므로, 답변은 **본문만 깔끔하게** 나와야 정상입니다.
 - 만약 생각이 보인다면 → STEP 0의 주소가 `:11500` 이 맞는지 다시 확인하세요 (`:11434` 로 남아 있으면 누출됩니다).

STEP 4 — before/after 비교법 (raw :11434 vs 게이트웨이 :11500)

“왜 게이트웨이가 필요한가”를 같은 질문, 같은 모델로 직접 체험해 봅니다. 차이는 오직 포트 번호뿐입니다.

① 게이트웨이 거치지 않고 직접(raw) — :11434

```
curl -s http://121.138.151.6:11434/api/chat -d '{
  "model": "qwen3.6:35b-a3b",
  "stream": false,
  "messages": [{"role": "user", "content": "외래 진료시간 알려줘"}]
}'
```

- 흔한 증상: 답변 안에 `<think>` 사용자가 진료시간을 물었다... `</think>` 같은 **흔한 말이 새어 나오거나**, 답이 끝없이 길어지거나, 빈 답이 옵니다. (모델이 “생각 모드”로 돌기 때문)

② 게이트웨이 경유 — :11500 (포트만 다름)

```
curl -s http://121.138.151.6:11500/api/chat -d '{
  "model": "qwen3.6:35b-a3b",
  "stream": false,
  "messages": [{"role": "user", "content": "외래 진료시간 알려줘"}]
}'
```

- 결과: **본문만 깔끔하게** 나옵니다. 생각 토큰 없음, 길이도 적당(상한 자동 적용).

무엇을 비교하나

비교 항목	raw :11434	게이트웨이 :11500
<code><think></code> 생각 누출	자주 새어 나옴	없음(자동 차단)
답변 길이	들쭉날쭉·과도하게 김	상한 안에서 적당
모델명 민감도	정확히 일치해야 함	조금 달라도 정규화

핵심: **모델도 같고 질문도 같은데 결과가 다릅니다.** 차이는 “게이트웨이가 운영 처리를 해주느냐”뿐입니다. 이것이 오늘의 한 줄(11434 → 11500)이 가진 힘입니다.

📌 **강사가 미리 채점했습니다 — “거절 규칙”이 품질을 가릅니다.** 여러분 챗봇 4개에 실제로 질문을 넣어 목적별로 채점했습니다(감염관리 5.0 · 의요기기 4.83 · 콜센터 4.75 · 진료통계 3.5 / 5점). 점수를 가른 건 정확도가 아니라 **“모를 때 거절하는 규칙”을 코드에 박았는가**였습니다. - 거절 규칙이 확실한 **감염관리**: “코로나19 격리기간?”(자료에 없음) → **확인되지 않습니다**, 환자 식별정보 → 즉시 차단 → **무결(5.0)**. - 거절을 모델에 맡긴 **진료통계**: “안과 외래 건수?”(없

는 진료과) → 거절 대신 *영똥한 일별* 수치를 지어냄 → 이 문항만 **1.0.** → 오늘 바꾸는 “주소 한 줄”과 함께, **“자료에 없으면 확인되지 않습니다”**라는 거절 규칙을 시스템 프롬프트에 꼭 박아 두세요. 그것이 하네스의 핵심입니다.

검진팀 실습 안내 (요약)

검진팀은 “주소 바꾸기”가 아니라 **AI가 없던 앱에 AI 기능 한 개를 새로 붙입니다**. 큰 흐름만 공통으로 안내합니다(상세 단계·복붙 코드는 검진팀 개인별 가이드에).

- **대상 앱 2개**: ① 제안서 생성기(제안서 마법사에 “AI 문구 생성” 버튼) ② 스케줄 관리(일정 표에 “질문하기”).
- **여러분이 할 일은 두 가지뿐**:
 1. 게이트웨이를 부르는 **연결 코드 한 덩어리 붙여넣기** (이 앱들은 외부 라이브러리 없이 파이썬 기본 `urllib` 로 게이트웨이 `:11500` 을 호출).
 2. 모델에게 줄 **프롬프트(규칙 + 맥락) 채우기** ← 이게 오늘의 진짜 실습.
- **게이트웨이가 `think:false`·길이 상한을 처리하므로**, 여러분은 프롬프트에만 집중하면 됩니다 = 하네스 실습.
- **⚠ 스케줄 앱 주의**: 직원 실명·사번 등이 들어간 데이터(`staff-profiles.json`)는 **더미(가짜)로만** 연습하세요.

안전수칙 (전원 공통, 필독)

- **환자정보·대외비 입력 금지.** 주민번호·환자번호·진단명·실명·사번 등은 **절대 입력창/프롬프트에 넣지 마세요.** 테스트는 **더미(가짜)·비식별 데이터**로만 합니다. (예: 고객사 **가나물산**, 환자 **홍길동**, “외래 진료시간 알려줘”)
- **공유 GPU는 1개입니다.** **:11500** 은 모두가 같이 쓰는 서버 1대입니다. 버튼 연타·대량 반복 질문·스트레스 테스트 금지. **질문 1번 = 모두가 줄 서는 1자리.** 답이 다 나온 뒤 다음 질문을 보내세요. 응답이 느리면 다른 사람이 쓰는 중일 수 있으니 잠시 기다립니다.
- **AI 답변은 참고용.** 숫자(요금·시간·항목 수·수가)는 최종 안내 전에 **실제 규정/엑셀로 사람이 한 번 더 확인**하세요. AI는 “문장 초안”만 담당합니다.
- **외부 모델(ChatGPT 등)에 실데이터 금지.** 오늘 쓰는 건 사내 공유 게이트웨이입니다. 대외비·임상 데이터를 외부 서비스에 넣지 마세요.
- **막히면 강사에게.** **/health** 가 안 오거나 생각 누출이 계속되면 혼자 끄꿍대지 말고 바로 알려 주세요(공유 서버 상태 점검). ``